U4 - Computer Abstractions and Technology

Translating and Starting a Program

A C program becomes a running program through **four major stages**:

1. **Compiler** → C to Assembly
2. **Assembler** → Assembly to Machine Code (Object File)
3. **Linker** → Combines object files into Executable
4. **Loader** → Loads program into memory and starts execution

1. Compiler

- Converts high-level C code into **assembly language** based on the target ISA.
- Produces symbolic machine instructions.
- Procedures can be compiled independently, reducing rebuild time.

2. Assembler

Role of the Assembler

- Converts assembly into **machine code**.
- Expands **pseudo-instructions** into actual instructions.
- Accepts numbers in various bases (binary, decimal, hex).
- Tracks labels and fills in their addresses using a **symbol table**.
- Produces an **object file**.

3. Linker

Why a Linker is Needed

- Recompiling the entire program for every change is wasteful.
- Standard library routines rarely change, so recompiling them repeatedly is inefficient.
- Linker enables **separate compilation**: only changed procedures are recompiled.

What the Linker Does

- Combines independent object files.
- Uses **relocation information** + **symbol tables** to resolve:
  - Procedure calls
  - Accesses to global data
- Patches instructions with final memory addresses.

Object File Contents

Each object file contains:

- Text (code) size and Data size
- Text and Data segments
- **Relocation information** (runtime address adjustments)
- **Symbol table** (names + addresses of symbols defined or referenced)

4. Loader

A system program that prepares an executable for execution.

Loader actions:

1. Reads executable header (gets text & data sizes).
2. Allocates an address space in memory.
3. Copies instructions and data into memory.
4. Copies program arguments onto the stack.
5. Initializes registers and stack pointer.
6. Jumps to a startup routine which:
   - Loads arguments into registers
   - Calls `main()`
   - On return, terminates program using exit system call

Static vs Dynamic Linking

Statically Linked Libraries (SLL)

- Library code is **included in the executable**.
- Executable becomes **larger**.
- If the library updates, the program still uses the **old version**.

Dynamically Linked Libraries (DLL)

- Library routines are **linked at runtime**, not at compile time.
- Executable size is **smaller**.
- Program + library store metadata so loader can find external routines.
- Loader runs a **dynamic linker** to update external references when the program starts.

Comparison Table

Feature	Statically Linked Libraries (SLL)	Dynamically Linked Libraries (DLL)
Linking time	Compile / Link time	Runtime
Library code included?	Yes	No
Executable size	Larger	Smaller
Library updates	Program keeps old version	Program can use updated library
Linking mechanism	Linker	Loader + dynamic linker

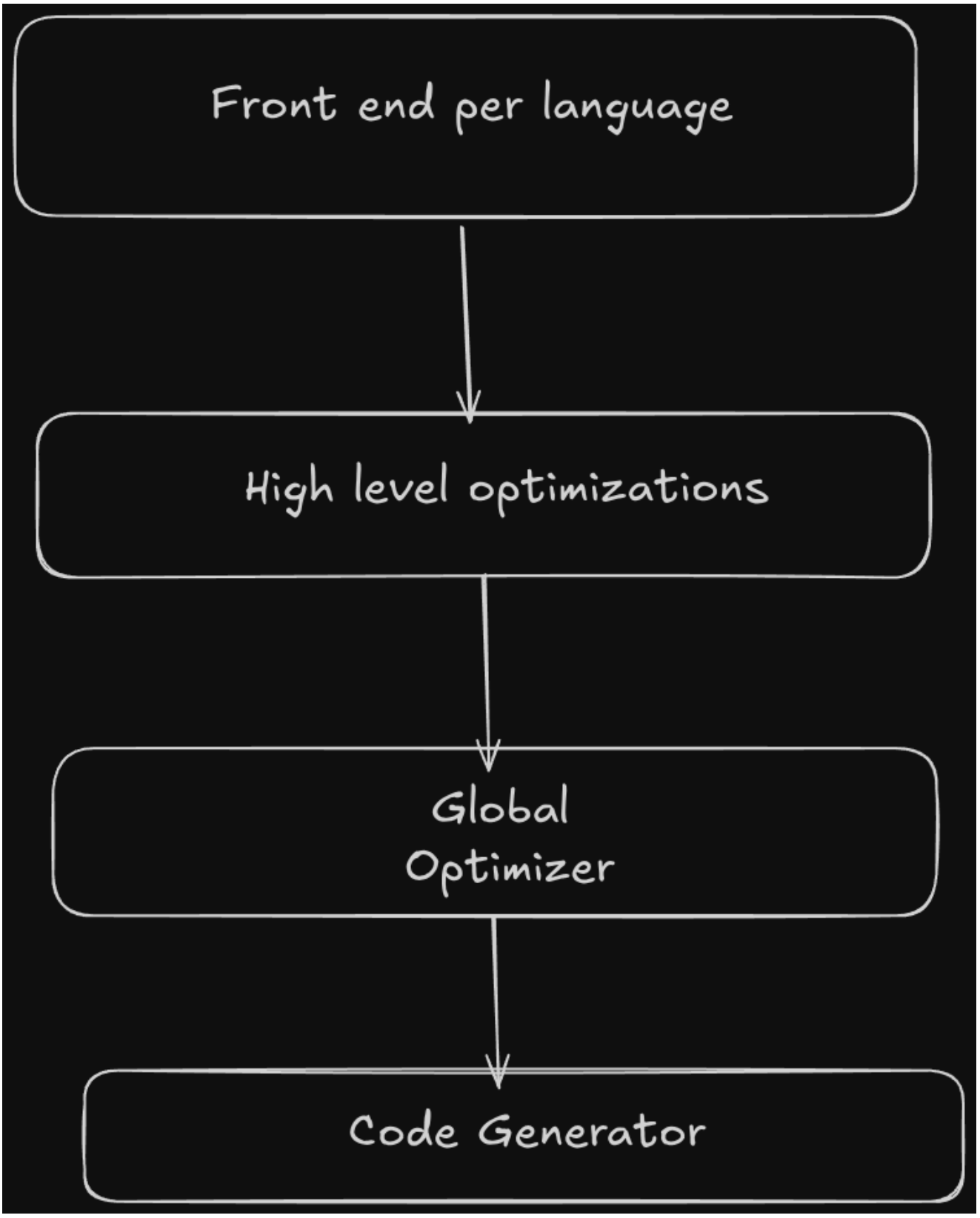
Compiling C

Modern compilers transform high-level programs (like C) into optimized machine code through multiple well-defined stages. These stages range from *language-specific* to *highly machine-dependent*.

Internal Anatomy of a Compiler

A complete compiler pipeline has four major conceptual regions:

Stage	Language Dependency	Machine Dependency	Purpose
Front End	High	Low	Parse and validate program; produce IR
High-Level Optimization	Medium	Low	Perform optimizations on IR independent of machine details
Low-Level Optimization	Low	Medium	Register allocation; IR cleanup; prepare for codegen
Back End	Low	High	Instruction selection; machine-dependent optimization



1. The Front End

The front end ensures the source program is **syntactically and semantically valid**, and translates it into an **intermediate representation (IR)**.

Front end stages:

1. Scanning & Lexical Analysis
2. Parsing (Syntax Analysis)

3. Semantic Analysis
4. IR Generation

1.1 Scanning & Lexical Analysis

Scanning

Removes non-token elements:

- comments
- extra whitespace
- preprocessing noise

Lexical Analysis

Converts characters into **tokens**, the basic vocabulary of the language.

Tokens include:

- identifiers
- keywords (`while` , `int` , `return`)
- operators (`+` , `==` , `+=`)
- literals (`1` , `3.14`)

Example

For code:

```
while (save[i] == k)
    i += 1;
```

Token sequence:

`while, (, save, [, i, ], ==, k, ), i, +=, 1, ;`

1.2 Parsing (Syntax Analysis)

Parsing checks whether the program follows the grammar of the language.
It builds a **Parse Tree (Syntax Tree)** using grammar rules.

If the parser cannot construct a valid tree → **syntax error**.

Example rule idea (not exact C grammar):

`while_stmt → "while" "(" expr ")" stmt`

If the code matches the rule structure, it is syntactically valid.

1.3 Semantic Analysis

Semantic analysis verifies the *meaning* of the program.

It performs:

- **Type checking** (e.g., adding int + float)
- **Variable declaration checks**
- **Label and flow-control correctness**
- Construction of the **symbol table**

The output is a **verified syntax tree** + **symbol table**.

Example:

```
int x;
x = "hello";   // semantic error: type mismatch
```

Semantically invalid, even though syntax is correct.

1.4 Intermediate Representation (IR) Generation

The IR is produced from the syntax tree.

Properties of IR:

- Machine-independent
- Language-independent after this point
- Easy to optimize

Modern IR looks like RISC-V but with **infinite virtual registers**:

Example IR:

```
r1 = load x
r2 = add r1, 5
r3 = call foo(r2)
```

Later, virtual registers (`r1` , `r2` , `r3`) get mapped to real machine registers.

2. Compiler Priorities

A good compiler tries to:

1. Produce **correct** code
2. Produce **fast** code
3. Minimize **code size**

Correctness always comes first.

3. High-Level Optimizations

These optimizations operate on IR, independent of machine details.

3.1 Procedure Inlining

Replaces a function call with the body of the function.

Example:

```
int square(int x) { return x*x; }

y = square(5);
```

After inlining:

```
y = 5 * 5;
```

Advantages:

- Removes function-call overhead
- Exposes new optimization opportunities

Disadvantages:

- Increases code size (especially for large functions)

3.2 Loop Unrolling

Reduces loop overhead by repeating the loop body multiple times.

Example:

```
for (i = 0; i < 4; i++)
    sum += a[i];
```

Unrolled:

```
sum += a[0];
sum += a[1];
sum += a[2];
sum += a[3];
```

Advantages:

- Faster (fewer branch instructions)

- Allows parallel execution

Disadvantages:

- Increases code size
- May increase register usage

3.3 Loop Interchange, Blocking, and similar

Used heavily in scientific computing to improve cache behavior.

4. Local & Global Optimizations

Local Optimization

Works **within a single basic block**.

Global Optimization

Works across **multiple blocks**, loops, or entire functions.

Register Allocation

Maps infinite virtual registers → finite physical registers.

Good register allocation is crucial for performance.

5. Key Optimization Techniques

5.1 Common Subexpression Elimination (CSE)

Find identical expressions; compute once.

Example:

```
a = x + y;
b = (x + y) * 3;
```

Optimized:

```
t = x + y;
a = t;
b = t * 3;
```

5.2 Strength Reduction

Replace expensive operations with cheaper ones.

Example:

```
x = i * 8;
```

Optimized:

```
x = i << 3; // shift instead of multiply
```

Shifts are cheaper than multiplication.

5.3 Constant Propagation + Constant Folding

Constant Propagation

Propagates constant values through the code.

Example:

```
x = 10;
y = x + 3;
```

Becomes:

```
y = 13;
```

Constant Folding

Evaluates constant expressions at compile time.

Example:

```
PI = 22/7; // replaced by 3.14
```

5.4 Copy Propagation

Eliminates unnecessary copies.

```
a = b;
c = a + 1;
```

Optimized:

```
c = b + 1;
```

5.5 Dead Code & Dead Store Elimination

Dead store example:

```
x = 5;
```

```
x = 7;      // store to x=5 is useless
```

Dead code example:

```
if (0)
{     printf("never executed");
}
```

Compilers remove such code.

5.6 Code Motion (Loop-Invariant Code Motion)

Moves computations that do not change inside a loop *out of the loop*.

Example:

```
for (i=0; i<n; i++)
    y = x * 10;     // invariant
```

Optimized:

```
y = x * 10;
for (i=0; i<n; i++)
    ;
```

5.7 Induction Variable Elimination

If multiple loop variables are related, remove the redundant ones.

Example concept:

```
i = 0;
j = 2*i;
while (i < n) {
    ... j used ...
    i++;
    j = 2*i;     // redundant
}
```

Replace uses of `j` with `2*i` and eliminate `j`.

6. Control Flow Graphs (CFG)

Global optimizations use **control flow graphs**:

- Nodes = basic blocks

- Edges = possible control flow

Transformations often reorder branches, move tests to reduce instructions, etc.

7. Code Generation (Back End)

Final stage:

- Select concrete machine instructions
- Perform machine-dependent optimizations
- Convert IR to assembly
- Assembler turns that into machine code

Modern compilers use **pattern matching** techniques (tree matchers) to select instructions efficiently.

8. Optimization Levels (GCC example)

- **-O1** → Medium optimization
- **-O2** → Full optimization
- **-O3** → Full + aggressive inlining and loop transforms

Optimization name	Explanation	gcc level
High level	***At or near the source level; processor independent***	
Procedure integration	Replace procedure call by procedure body	03
Local	***Within straight-line code***	
Common subexpression elimination	Replace two instances of the same computation by single copy	01
Constant propagation	Replace all instances of a variable that is assigned a constant with the constant	01
Stack height reduction	Rearrange expression tree to minimize resources needed for expression evaluation	01
Global	***Across a branch***	
Global common subexpression elimination	Same as local, but this version crosses branches	02
Copy propagation	Replace all instances of a variable ***A*** that has been assigned ***X*** (i.e., ***A*** = ***X***) with ***X***	02
Code motion	Remove code from a loop that computes the same value each iteration of the loop	02
Induction variable elimination	Simplify/eliminate array addressing calculations within loops	02
Processor dependent	***Depends on processor knowledge***	
Strength reduction	Many examples; replace multiply by a constant with shifts	01
Pipeline scheduling	Reorder instructions to improve pipeline performance	01
Branch offset optimization	Choose the shortest branch displacement that reaches target	01

Seven Great Ideas in Computer Architecture

1. **Abstraction** – Hide unnecessary details so humans don’t lose their minds.
2. **Pipelining** – Break work into stages, keep all stages busy.
3. **Parallelism** – Do multiple things at once where possible.
4. **Make the Common Case Fast** – Optimize what happens most often.

5. **Prediction** – Guess likely outcomes to avoid waiting.
6. **Hierarchy** – Organize memories and systems in layers.
7. **Dependability via Redundancy** – Add extra parts so failure doesn't kill the system.

Performance via Pipelining

Instruction Execution Stages

Fetch → Decode → Execute → Memory Access → Writeback.

Given operation times:

- Memory access: **200 ps**
- ALU: **200 ps**
- Register read/write: **100 ps**

Single-Cycle (Non-Pipelined) Processor

Clock must match **slowest instruction**, so:
Total cycle time:

$$T_{\text{clk}} = T_1 + T_2 + T_3 + T_4 + T_5$$

Program with **N** instructions:

$$T_{\text{total}} = N \times T_{\text{clk}}$$

Pipelined Processor

Each stage works in parallel on different instructions.
Processors may use 1, 2, 3, 5, 6... pipeline stages depending on design.

Sequential vs Parallel Computing

Sequential computing:
Operations happen one at a time on one processor.

Parallel computing:
Break program into smaller tasks that run simultaneously on multiple units.

Performance via Parallelism:
Architects design hardware to exploit parallel operations wherever possible.

Make the Common Case Fast

Speeding up a frequent operation gives a larger performance benefit than optimizing rare operations.
The “common case” is often simpler, which makes optimizing it easier too.

Amdahl’s Law

If part of a program cannot be parallelized, it limits total speedup no matter how many processors you throw at it.

General formula:

$$\text{Execution Time}_{\text{after improvement}} = \frac{\text{Execution Time}_{\text{affected}}}{\text{Amount of Improvement}} + \text{Execution Time}_{\text{unaffected}}$$

Example:
A program takes 20 hours, with **1 hour sequential** and **19 hours parallelizable**.

Fallacies and Pitfalls

Fallacy:
Assuming improving one part improves whole-system performance proportionally.

Pitfall:
Using an incomplete part of a performance equation as the whole metric.

Corollary example:
“Make the common case fast”, well-established and proven idea.

Memory Hierarchy

Register → Cache → Main Memory → Flash → Disk → Remote Storage

As you move **left → right**:

- Storage capacity **increases**
- Speed **decreases**
- Cost per bit **decreases**

Why memory matters:

- Memory speed affects performance.
- Memory capacity limits size of solvable problems.
- Memory often contributes most to system cost.

Abstraction in Systems

Software and hardware are layered.
Hardware sits deepest; applications outermost.

Operating System provides:

- I/O handling
- Memory + storage allocation
- Protection + sharing of resources

Compiler:
Converts high-level code → assembly.

Assembler:
Converts assembly → machine code.

Why Use Abstraction?

- Manages complexity
- Improves programmer productivity
- Allows programs to run across hardware systems
- Lets us think in natural notation (A + B instead of registers/memory ops)

Performance via Prediction

Branch instructions break pipelines.
The processor doesn't know which way a branch will go, so:

- It fetches the next instruction anyway.
- On mismatch, pipeline is **flushed**, wasted work.

Branch Prediction:
Guess branch direction → fetch correct next instruction → avoid flush penalty.

Higher accuracy → higher performance.

Dependability via Redundancy

Hardware fails.
We use redundancy so that:

- Other components take over when one fails
- Failures can be detected
- Increasing transistor density makes redundancy cheaper

Chip Manufacturing Process

Silicon (semiconductor) → doped with materials → becomes

- Conductor
- Insulator
- Switch (transistor)

Billions of these form a VLSI chip.

Steps

Silicon ingot → sliced → wafer → components patterned → tested → diced → packaged → tested → shipped.

Why wafers are circular?

Growing silicon crystal uses **cylindrical spinning**, which is most efficient → leads to circular wafers.

Cost of Chips

Dies per wafer:

$$\text{Dies per wafer} \approx \frac{\text{Wafer Area}}{\text{Die Area}}$$

(Aprox. because edges can’t fit full rectangular dies.)

Cost per die:

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

Die yield:

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}))^N}$$

where **N** = process-complexity factor.

Performance

Response Time (Execution Time)

The **total** time a computer takes to complete a task.
Includes: CPU time, memory access, disk I/O, OS overhead, etc.
This metric matters most to individual users.

Throughput (Bandwidth)

Number of tasks completed per unit time.

Case 1 - Faster Processor

Response time ↓ → Throughput ↑
Both improve.

Case 2 - Add More Processors for Independent Tasks

Each task still takes the same time → Response time unchanged.
But more tasks run at once → Throughput ↑

Why Performance Measurement Is Hard

We want **lower execution time** for a given task, but:

Performance ∝ 1 / Execution Time

So:

$$\text{Performance} = \frac{1}{\text{Execution Time}}$$

For two computers X and Y:

If

$$\text{Performance}_X > \text{Performance}_Y$$

then

$$\text{Execution Time}_X < \text{Execution Time}_Y$$

CPU Execution Time (CPU Time)

Time CPU spends computing the task, excluding waiting for I/O or other programs.

User CPU Time: time spent running your code.
System CPU Time: time spent in OS routines invoked by your program.

When we say "CPU performance", we usually mean **User CPU Time**.

CPU Execution Time Formula

Using clock cycle time:

$$\text{CPU Time} = \text{Clock Cycles} \times \text{Clock Cycle Time}$$

Using clock rate:

$$\text{CPU Time} = \frac{\text{Clock Cycles}}{\text{Clock Rate}}$$

Instruction Performance

Execution time must consider **number of instructions** and **CPI**.

Clock Cycles Formula

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{CPI}$$

CPI (Cycles Per Instruction): average cycles each instruction takes.

IPC (Instructions Per Cycle):

$$\text{IPC} = \frac{1}{\text{CPI}}$$

If IPC = 2 → CPI = 0.5.

Formula Sheet

Total Execution Time

Using clock cycle time T_{clk} :

$$\text{Execution Time} = \text{Clock Cycles} \times T_{\text{clk}}$$

Using clock frequency f_{clk} :

$$\text{Execution Time} = \frac{\text{Clock Cycles}}{f_{\text{clk}}}$$

Clock Cycles

$$\text{Clock Cycles} = \text{Instruction Count} \times \text{CPI}$$

Expanded Execution Time

Using CPI and frequency:

$$\text{Execution Time} = \text{Instruction Count} \times \frac{\text{CPI}}{f_{\text{clk}}}$$

Solve for Clock Frequency

$$f_{\text{clk}} = \frac{\text{Instruction Count} \times \text{CPI}}{\text{Execution Time}}$$

Solve for CPI

Using clock frequency:

$$\text{CPI} = \frac{\text{Execution Time}}{\text{Instruction Count}} \times f_{\text{clk}}$$

Using clock cycle time:

$$\text{CPI} = \frac{\text{Execution Time}}{\text{Instruction Count} \times T_{\text{clk}}}$$

Multiple Instruction Classes

If different types have different CPIs:

$$\text{Clock Cycles} = \sum_i (\text{Instruction Count}_i \times \text{CPI}_i)$$

Thus:

$$\text{Execution Time} = \sum_i (\text{Instruction Count}_i \times \text{CPI}_i \times T_{\text{clk}})$$

Instructions Per Second

$$\text{IPS} = \text{IPC} \times f_{\text{clk}}$$

Performance of a Program

The performance of a program depends on **five major factors**:
algorithm, programming language, compiler, ISA (instruction set architecture), and hardware.

1. Algorithm

Affects:

- **Instruction Count (IC):** Different algorithms perform different operations → different numbers of instructions executed.
- **CPI:** Some algorithms rely heavily on operations that have higher latency (e.g., divides), impacting cycles per instruction.

2. Programming Language

Affects:

- **Instruction Count:** High-level statements translate to different amounts of machine instructions.
- **CPI:** Languages with lots of abstraction (Java, Python) may require extra indirection, method lookups, or checks → higher CPI instructions.

3. Compiler

Affects:

- **Instruction Count:** A good compiler reduces unnecessary instructions and optimizes structure.
- **CPI:** Compiler decides which instructions to use; better instruction selection lowers CPI.

Compilers are the “translator-artists” deciding how clean or messy your final machine code is.

4. Instruction Set Architecture (ISA)

Affects:

- **Instruction Count:** Some architectures need more instructions to express the same computation.
- **CPI:** Different instructions have inherently different latencies.
- **Clock Rate:** The ISA influences how complex the hardware must be, which can limit how fast the clock can run.

ISA = the contract between software and hardware → affects all layers of performance.

5. Hardware (Microarchitecture)

Affects:

- **CPI** (pipeline depth, cache sizes, branch predictors)
- **Clock Rate**
- **Instruction Count** (via microarchitectural features that allow parallel execution)

Turbo Mode

Modern CPUs (e.g., Intel Core i7) dynamically increase clock rate (≈10%) when thermal headroom allows.
Clock rate is no longer fixed → when analyzing performance, use **average clock rate**.

The Multicore Shift

Single-core performance hit barriers (power wall, frequency scaling limits).
Solution: **multiple cores per chip**, creating **multicore** processors ("quad-core," "octa-core," etc.).

Challenges for Programmers

Parallel programming is hard because:

- Work must be divided so each core has similar load.
- Synchronization and coordination introduce overhead.
- Many tasks are not naturally parallel.

Parallelism improves **throughput** more than **response time**.

Areas of Parallelism

Parallelism pops up in several architectural layers:

1. Synchronization

Coordinating tasks across threads/cores.

2. Subword Parallelism (SIMD)

Using vector instructions to operate on multiple data elements at once.

3. Instruction-Level Parallelism (ILP)

Hardware techniques (pipelining, out-of-order execution) running multiple instructions concurrently.

4. Memory Hierarchy Parallelism

- **Cache coherence:** Keeping data consistent across cores' caches.
- **RAID:** Parallelism across multiple storage disks.

Benchmarking the Intel Core i7

What is a Benchmark?

A *workload* is simply a set of programs you run on a machine to see how it behaves. To compare two systems, you can compare how long each takes to run the same workload.

A *benchmark* is just a workload someone has carefully selected to act as a proxy for "real" work.
If the benchmark behaves similarly to your actual programs, it becomes a decent predictor of performance.

SPEC: System Performance Evaluation Cooperative

SPEC is a consortium of vendors who agreed on standardized benchmark suites.
The idea is simple: if everyone uses the same tests, you can compare hardware without the "my-custom-benchmark beats your-custom-benchmark" drama.

SPEC CPU Benchmarks

SPEC's flagship suite focuses on CPU and memory system performance.

- The earliest version (1989) was **SPEC89**.
- Today, the widely used one is **SPEC CPU2017**.

SPEC CPU2017 contains:

- **10 integer benchmarks** (SPECspeed 2017 Integer)
- **13 floating-point benchmarks** (SPECspeed 2017 FP)

These aren't toy programs. They include:

- Compilers
- Chess engines
- Quantum simulation kernels
- Finite-element structural analysis
- Molecular dynamics particle codes
- Sparse linear algebra used in fluid dynamics

Each benchmark stresses different parts of the processor pipeline: branching, integer ALU, floating-point units, memory hierarchy, and so on.

Why Use SPEC for Intel Core i7?

The Core i7 is a modern superscalar, out-of-order, multicore design with heavy dependence on:

- branch prediction

- speculative execution
- cache hierarchy
- instruction-level parallelism

SPEC workloads poke all these architectural pressure points.
That means the Core i7's SPEC results reflect real CPU behavior much better than synthetic "loop of adds" benchmarks.

What SPEC Measures

SPEC traditionally reports two kinds of metrics:

- **Speed** (SPECspeed): how fast a *single* program runs
- **Throughput** (SPECrate): how many copies of the program the system can run in parallel

Speed is about response time; rate is about throughput.
A Core i7 with multiple cores can shine in throughput even if a single core isn't earth-shattering.

The Power Wall

Modern integrated circuits are built using **CMOS** (Complementary Metal-Oxide-Semiconductor) technology. In CMOS, most of the energy is consumed **only when transistors switch states**.
This is called **dynamic energy**, the cost of charging and discharging tiny capacitors inside each gate.

Dynamic Energy

Each switching event must charge or discharge a load capacitance.
The energy needed for a full logic cycle (0 → 1 → 0 or 1 → 0 → 1) is proportional to:

$$E \propto C \cdot V^2$$

For a *single* transition (just 0 → 1 or 1 → 0):

$$E = \tfrac{1}{2} C V^2$$

Dynamic Power

Power is simply the rate at which energy is consumed.
If a circuit switches at frequency fff, then:

$$P = \tfrac{1}{2} C V^2 f$$

This means power grows with:

- higher capacitive loads (bigger, more complex circuits),
- higher voltages,
- and higher clock frequencies.

Why We Hit the “Power Wall”

Historically, performance increased by pushing frequency higher.
But because power scales *linearly* with f and *quadratically* with V, cranking clock rates led to chips that ran too hot to cool efficiently.

Past a point, designers simply could not raise clock speeds, this thermal limit is what we call the **Power Wall**.

Once industry slammed into it, the strategy shifted from faster single cores to **multicore architectures**, where parallelism replaced raw clock scaling.

If this helped you, treat me with a dosa.